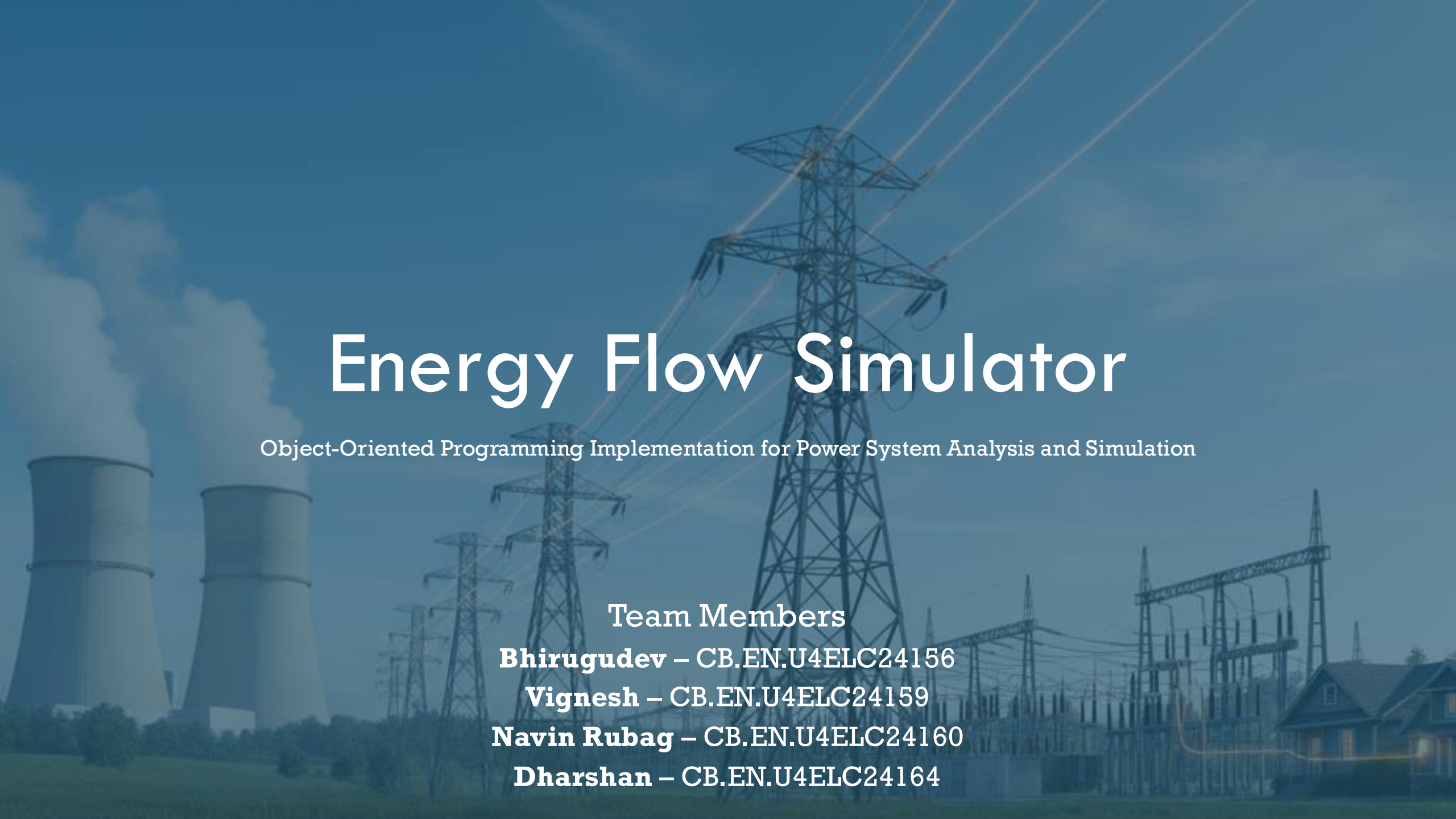




Energy Flow Simulator

Object-Oriented Programming Implementation for Power System Analysis and Simulation

Team Members

Bhirugudev – CB.EN.U4ELC24156

Vignesh – CB.EN.U4ELC24159

Navin Rubag – CB.EN.U4ELC24160

Dharshan – CB.EN.U4ELC24164

Contents

- 01. Introduction
- 02. Theory & Concepts
- 03. Code Flow & Process
- 04. OOP Implementation
- 05. Output & Results
- 06. Project Outcomes



What is an Energy Flow Simulator?

Software Power System Model :

A comprehensive software model that simulates the complete **lifecycle of electrical energy** from **generation to consumption** without requiring physical hardware.

Generation Analysis:

Models **power production** from **thermal, hydro, solar, wind, and nuclear sources** with realistic constraints.

System Efficiency:

Analyzes **power flow**, calculates **losses**, and determines overall **system efficiency** for optimization

Power System Theory

Generation:

1. Produces power from thermal, hydro, solar, wind, and nuclear sources.
2. Each source has unique characteristics and efficiency factors.
3. Generation capacity varies based on resource availability

Transmission:

1. Transfers power using high-voltage lines with resistance losses
2. Includes reactance losses and voltage drop calculations
3. Optimizes power transfer efficiency across distances

Distribution & Utilization:

1. Steps down the voltage via transformers for safe delivery
2. Supplies power through feeder networks to consumers
3. Represents consumer energy consumption patterns



Code Flow Process

01

Input Data Collection

- Gather consumer data, environmental factors (solar irradiance & wind speed), system voltage levels, and component efficiency parameters, The energy requirements and consumer details

02

Power Generation

- Generate power from multiple sources using specific algorithms for thermal, hydro, solar, wind, and nuclear plant models.

03

Transmission & Losses

- Transmit generated power through high-voltage lines while calculating resistance and reactance losses in real-time.

04

Distribution Network

- Step down voltage through transformers and distribute power via feeder networks to reach end consumers.

05

Output & Visualization

- Display simulation results including power flows, voltage profiles, and system efficiency metrics for analysis.

OOP Concepts Implementation

6

Core Concepts

Inheritance

Child classes inherit from the base Generator class for code reuse and hierarchy maintenance

Thermal Plant, Hydro Plant, Solar Plant inherit from Generator.

Reuses common logic and maintains a clean hierarchy.

Enables polymorphic behavior across plant types

7

OOP Principles

Abstraction

Generator ABC hides complex formulas and provides clean interface through abstract methods.

Abstract `generate_power()` method in base class

Hides internal complexity from users.

Provides consistent interface across all generators

100%

Error Handling

Auto

Reports

Polymorphism

Each generator overrides `generate_power()` with source-specific implementations.

Thermal uses efficiency factors.

Solar uses irradiance calculations.

Wind uses wind speed factors.

Advanced OOP Features & Output

Encapsulation & Composition:

Each class maintains private data and the Grid. The operator composes the entire power system through object relationships.

Exception Handling:

Try-except blocks in every section
Safe runtime error management
Graceful degradation on failures

File & Data Handling:

Automatic report generation in .txt format
Matplotlib visualization for power flow graphs
Comprehensive data export capabilities

System Outputs:

Total demand, generation, losses, and efficiency metrics

Project Success:

Full OOP implementation with real engineering application

Learning Outcomes:

Demonstrates practical OOP concepts in energy systems

```
#!/usr/bin/env python3
"""
Energy Flow Simulator - Complete Version
With all segments: Generation, Transmission, Distribution, Utilization
"""

import math, sys, os
from datetime import datetime
from abc import ABC, abstractmethod
import matplotlib.pyplot as plt
import numpy as np

# =====
# UTILITIES
# =====

def safe_float(prompt, min_val=None, max_val=None):
    while True:
        try:
            v = float(input(prompt).strip())
            if min_val is not None and v < min_val:
                print(f"Value must be >= {min_val}")
                continue
            if max_val is not None and v > max_val:
                print(f"Value must be <= {max_val}")
                continue
            return v
        except Exception as e:
            print(f"Error handled in safe_float: {e}")

def safe_int(prompt, min_val=None, max_val=None):
```

```
while True:
    try:
        v = int(input(prompt).strip())
        if min_val is not None and v < min_val:
            print(f"Value must be >= {min_val}")
            continue
        if max_val is not None and v > max_val:
            print(f"Value must be <= {max_val}")
            continue
        return v
    except Exception as e:
        print(f"Error handled in safe_int: {e}")

def timestamp():
    return datetime.now().strftime("%Y%m%d_%H%M%S")

def ensure_dir(d):
    if not os.path.isdir(d):
        os.makedirs(d, exist_ok=True)

def mw_to_kw(mw): return mw * 1000.0
def kw_to_mw(kw): return kw / 1000.0
def i_from_power_mw(power_mw, voltage_kv, pf=1.0):
    if voltage_kv <= 0 or pf == 0:
        return 0.0
    return (power_mw * 1e6) / (math.sqrt(3) * voltage_kv * 1e3 * pf)
```

```
# SEGMENT 1: GENERATION SYSTEM
# =====

class Generator(ABC):
    def __init__(self, name, capacity_mw, efficiency=0.9, voltage_kv=11.0, is_on=True):
        self.name = name
        self.capacity_mw = float(capacity_mw)
        self.efficiency = float(efficiency)
        self.voltage_kv = float(voltage_kv)
        self.is_on = bool(is_on)

    @abstractmethod
    def generate_power(self, requested_mw, **kwargs):
        pass

    def turn_off(self): self.is_on = False
    def turn_on(self): self.is_on = True

class ThermalPlant(Generator):
    def generate_power(self, requested_mw, **kwargs):
        if not self.is_on:
            return {"output_mw": 0.0, "internal_loss_mw": 0.0}
        try:
            available = min(self.capacity_mw, requested_mw)
            output = available * self.efficiency
            return {"output_mw": output, "internal_loss_mw": available - output}
        except Exception as e:
            print(f"Error handled in ThermalPlant generation: {e}")
            return {"output_mw": 0.0, "internal_loss_mw": 0.0}
```

```
class HydroPlant(Generator):
    def generate_power(self, requested_mw, **kwargs):
        if not self.is_on:
            return {"output_mw": 0.0, "internal_loss_mw": 0.0}
        try:
            available = min(self.capacity_mw, requested_mw)
            output = available * max(0.85, self.efficiency)
            return {"output_mw": output, "internal_loss_mw": available - output}
        except Exception as e:
            print(f"Error handled in HydroPlant generation: {e}")
            return {"output_mw": 0.0, "internal_loss_mw": 0.0}

class SolarPlant(Generator):
    def generate_power(self, requested_mw, irradiance_factor=1.0, **kwargs):
        if not self.is_on:
            return {"output_mw": 0.0, "internal_loss_mw": 0.0}
        try:
            available = min(self.capacity_mw * irradiance_factor, requested_mw)
            output = available * max(0.0, self.efficiency * 4.5)
            return {"output_mw": output, "internal_loss_mw": max(0.0, available - output)}
        except Exception as e:
            print(f"Error handled in SolarPlant generation: {e}")
            return {"output_mw": 0.0, "internal_loss_mw": 0.0}

class WindFarm(Generator):
    def generate_power(self, requested_mw, wind_factor=1.0, **kwargs):
        if not self.is_on:
            return {"output_mw": 0.0, "internal_loss_mw": 0.0}
        try:
```

```

try:
    available = min(self.capacity_mw * wind_factor, requested_mw)
    output = available * max(0.25, self. efficiency)
    return {"output_mw": output, "internal_loss_mw": max(0.0, available - output)}
except Exception as e:
    print(f"Error handled in WindFarm generation: {e}")
    return {"output_mw": 0.0, "internal_loss_mw": 0.0}

class NuclearPlant(Generator):
    def generate_power(self, requested_mw, **kwargs):
        if not self.is_on:
            return {"output_mw": 0.0, "internal_loss_mw": 0.0}
        try:
            available = min(self.capacity_mw, requested_mw)
            output = available * max(0.9, self. efficiency)
            return {"output_mw": output, "internal_loss_mw": available - output}
        except Exception as e:
            print(f"Error handled in NuclearPlant generation: {e}")
            return {"output_mw": 0.0, "internal_loss_mw": 0.0}

# =====
# SEGMENT 2: TRANSMISSION SYSTEM
# =====

class TransmissionLine:
    def __init__(self, name, length_km, r_per_km, x_per_km, voltage_kv=220.0, capacity_mva=1000.0):
        self.name = name

```

```

        self.length_km = float(length_km)
        self.r_per_km = float(r_per_km)
        self.x_per_km = float(x_per_km)
        self.voltage_kv = float(voltage_kv)
        self.capacity_mva = float(capacity_mva)

    def total_resistance(self):
        return self.r_per_km * self.length_km

    def total_reactance(self):
        return self.x_per_km * self.length_km

    def calc_line_loss_mw(self, power_mw, pf=1.0):
        try:
            I = i_from_power_mw(power_mw, self.voltage_kv, pf)
            R = self.total_resistance()
            return (I ** 2) * R / 1e6
        except Exception as e:
            print(f"Error handled in transmission loss calculation: {e}")
            return 0.0

    def voltage_drop(self, power_mw, pf=1.0):
        try:
            I = i_from_power_mw(power_mw, self.voltage_kv, pf)
            R = self.total_resistance()
            X = self.total_reactance()
            drop = I * math.sqrt((R**2 + X**2)) / 1000.0

```

```

        return drop
    except Exception as e:
        print(f"Error handled in voltage drop calculation: {e}")
        return 0.0

def transmit(self, power_mw, pf=1.0):
    try:
        loss = self.calc_line_loss_mw(power_mw, pf)
        received = max(0.0, power_mw - loss)
        v_drop = self.voltage_drop(power_mw, pf)
        return {"sent_mw": power_mw, "loss_mw": loss, "received_mw": received, "voltage_drop_kv": v_drop}
    except Exception as e:
        print(f"Error handled in transmission: {e}")
        return {"sent_mw": power_mw, "loss_mw": 0.0, "received_mw": power_mw, "voltage_drop_kv": 0.0}

# =====
# SEGMENT 3: DISTRIBUTION SYSTEM
# =====

class DistributionSystem:
    def __init__(self, name, input_voltage_kv=132.0, output_voltage_kv=11.0, transformer_efficiency=0.98):
        self.name = name
        self.input_voltage_kv = float(input_voltage_kv)
        self.output_voltage_kv = float(output_voltage_kv)
        self.transformer_efficiency = float(transformer_efficiency)
        self.feeders = []

```

```

    def add_feeder(self, feeder_name, load_mw):
        try:
            self.feeders.append({"name": feeder_name, "load_mw": float(load_mw)})
        except Exception as e:
            print(f"Error handled adding feeder: {e}")

    def step_down(self, power_mw):
        try:
            out = power_mw * self.transformer_efficiency
            return {"out_mw": out, "transformer_loss_mw": power_mw - out}
        except Exception as e:
            print(f"Error handled in step-down transformation: {e}")
            return {"out_mw": power_mw, "transformer_loss_mw": 0.0}

    def allocate_loads(self, available_mw):
        try:
            total_demand = sum(f["load_mw"] for f in self.feeders)
            if total_demand == 0:
                return []
            return [{"feeder": f["name"], "allocated_mw": (f["load_mw"] / total_demand) * available_mw}
                    for f in self.feeders]
        except Exception as e:
            print(f"Error handled in load allocation: {e}")
            return []

    def total_demand(self):
        try:
            return sum(f["load_mw"] for f in self.feeders)

```

```

    except Exception as e:
        print(f"Error handled calculating total demand: {e}")
        return 0.0

def auto_adjust_feeders(self, new_total_demand):
    try:
        current_total = self.total_demand()
        if current_total == 0:
            for feeder in self.feeders:
                feeder["load_mw"] = new_total_demand / len(self.feeders)
        else:
            scale_factor = new_total_demand / current_total
            for feeder in self.feeders:
                feeder["load_mw"] *= scale_factor
    except Exception as e:
        print(f"Error handled in automatic feeder adjustment: {e}")

# =====
# SEGMENT 4: UTILIZATION SYSTEM (CONSUMERS)
# =====

class Consumer:
    consumer_registry = []

    def __init__(self, consumer_id, consumer_type, power_rating_kw, daily_hours):
        self.consumer_id = consumer_id
        self.consumer_type = consumer_type

```

```

        self.power_rating_kw = float(power_rating_kw)
        self.daily_hours = float(daily_hours)
        Consumer.consumer_registry.append(self)

    def daily_energy_kwh(self):
        try:
            return self.power_rating_kw * self.daily_hours
        except Exception as e:
            print(f"Error handled in daily energy calculation: {e}")
            return 0.0

    def peak_power_kw(self):
        return self.power_rating_kw

# =====
# GRID OPERATOR - MANAGES ALL SEGMENTS
# =====

class GridOperator:
    def __init__(self):
        self.generators = []
        self.lines = []
        self.distributions = []
        self.logs = []
        self.simulation_history = []
        self._setup_default_infrastructure()

```

```

def _setup_default_infrastructure(self):
    try:
        # GENERATION SEGMENT setup
        self.generators = [
            ThermalPlant("Thermal_A", 500, 0.85),
            ThermalPlant("Thermal_B", 400, 0.88),
            HydroPlant("Hydro_C", 300, 0.9),
            SolarPlant("Solar_PV", 200, 0.18),
            WindFarm("Wind_Farm", 150, 0.35),
            NuclearPlant("Nuclear_D", 800, 0.92)
        ]

        # TRANSMISSION SEGMENT setup
        self.lines = [
            TransmissionLine("Line_A", 150, 0.05, 0.08, 220, 800),
            TransmissionLine("Line_B", 300, 0.04, 0.06, 400, 1500),
            TransmissionLine("Line_C", 80, 0.06, 0.09, 132, 500)
        ]

        # DISTRIBUTION SEGMENT setup
        dist = DistributionSystem("City_Distribution", 220, 11, 0.985)
        dist.add_feeder("Residential", 50)
        dist.add_feeder("Commercial", 40)
        dist.add_feeder("Industrial", 60)
        dist.add_feeder("Public_Services", 20)
        self.distributions = [dist]
    except Exception as e:
        print(f"Error handled in infrastructure setup: {e}")

```

```

def total_generation_capacity(self):
    try:
        return sum(g.capacity_mw for g in self.generators if g.is_on)
    except Exception as e:
        print(f"Error handled calculating generation capacity: {e}")
        return 0.0

def auto_adjust_system(self, new_total_demand):
    try:
        # Adjust DISTRIBUTION SEGMENT
        for dist in self.distributions:
            dist.auto_adjust_feeders(new_total_demand)

        # Adjust GENERATION SEGMENT
        current_capacity = self.total_generation_capacity()
        if new_total_demand > current_capacity:
            for gen in sorted(self.generators, key=lambda x: -x.efficiency):
                if not gen.is_on and current_capacity < new_total_demand:
                    gen.turn_on()
                    current_capacity += gen.capacity_mw
                    self.logs.append(f"Auto-activated {gen.name}")

            self.logs.append(f"System adjusted for {new_total_demand:.2f} MW")
    except Exception as e:
        print(f"Error handled in system auto-adjustment: {e}")

def dispatch_merit_order(self, demand_mw, **kwargs):
    try:

```

```

try:
    remaining = demand_mw
    dispatch = []

    # GENERATION SEGMENT dispatch
    for g in sorted(self.generators, key=lambda x: (-x. efficiency, -x.capacity_mw)):
        if remaining <= 0:
            if g.is_on:
                g.turn_off()
                continue

            if not g.is_on:
                g.turn_on()

        request = min(g.capacity_mw, remaining)
        result = g.generate_power(request, **kwargs)
        dispatch.append({
            "gen": g.name,
            "requested_mw": request,
            "supplied_mw": result["output_mw"],
            "internal_loss_mw": result["internal_loss_mw"]
        })
        remaining -= result["output_mw"]

    return {"dispatch": dispatch, "shortfall_mw": max(0.0, remaining)}
except Exception as e:
    print(f"Error handled in merit order dispatch: {e}")

```

```

363         return {"dispatch": [], "shortfall_mw": demand_mw}
364
365     def choose_lines_for_power(self, required_mw):
366         try:
367             # TRANSMISSION SEGMENT line selection
368             used = []
369             remaining = required_mw
370
371             for L in sorted(self.lines, key=lambda x: (-x.voltage_kv, -x.capacity_mva)):
372                 if remaining <= 0:
373                     break
374                 can_send = min(remaining, L.capacity_mva * 0.8)
375                 res = L.transmit(can_send)
376                 used.append((L, res))
377                 remaining -= res["sent_mw"]
378
379             return used
380         except Exception as e:
381             print(f"Error handled in line selection: {e}")
382             return []
383
384     def distribute_to_feeders(self, distribution, available_mw):
385         try:
386             # DISTRIBUTION SEGMENT power distribution
387             step = distribution.step_down(available_mw)
388             allocations = distribution.allocate_loads(step["out_mw"])
389             return {"transformer": step, "allocations": allocations}

```

```

except Exception as e:
    print(f"Error handled in distribution: {e}")
    return {"transformer": {"out_mw": 0, "transformer_loss_mw": 0}, "allocations": []}

def system_efficiency_and_losses(self, dispatch_results, line_usages, distribution_reports):
    try:
        # Calculate losses across all segments
        gen_loss = sum(d["internal_loss_mw"] for d in dispatch_results) # GENERATION loss
        tx_loss = sum(ru["loss_mw"] for (_, ru) in line_usages) # TRANSMISSION loss
        dist_loss = sum(d["transformer"]["transformer_loss_mw"] for d in distribution_reports) # DISTRIBUTION loss

        total_supplied = sum(d["supplied_mw"] for d in dispatch_results)
        total_loss = gen_loss + tx_loss + dist_loss
        efficiency = (total_supplied / (total_supplied + total_loss)) * 100 if (total_supplied + total_loss) > 0 else 0

        return {
            "gen_loss_mw": gen_loss,
            "tx_loss_mw": tx_loss,
            "dist_loss_mw": dist_loss,
            "total_loss_mw": total_loss,
            "system_efficiency_pct": efficiency,
            "total_supplied_mw": total_supplied,
            "voltage_drops": [res["voltage_drop_kv"] for (_, res) in line_usages]
        }
    except Exception as e:
        print(f"Error handled in efficiency calculation: {e}")
        return {"gen_loss_mw": 0, "tx_loss_mw": 0, "dist_loss_mw": 0, "total_loss_mw": 0,

```

```

            "system_efficiency_pct": 0, "total_supplied_mw": 0, "voltage_drops": []}

    def reset_logs(self):
        self.logs = []

# =====
# REPORTING & VISUALIZATION SYSTEM
# =====

REPORT_DIR = "reports"
PLOT_DIR = "plots"
ensure_dir(REPORT_DIR); ensure_dir(PLOT_DIR)

def save_report(filename, content_lines):
    try:
        path = os.path.join(REPORT_DIR, filename)
        with open(path, "w") as f:
            f.write("\n".join(content_lines))
        print(f"Report saved: {path}")
    except Exception as e:
        print(f"Error handled saving report: {e}")

def create_all_plots(grid, dispatch, line_usages, distributions, timestamp_tag):
    plot_files = {}

    try:
        # PLOT 1: Generation Mix Pie Chart

```

```

active_gens = [g for g in grid.generators if g.is_on]
if active_gens:
    plt.figure(figsize=(8, 6))
    sizes = [g.capacity_mw for g in active_gens]
    labels = [f"{g.name}\n({g.capacity_mw} MW)" for g in active_gens]
    plt.pie(sizes, labels=labels, autopct='%1.1f%%', startangle=90)
    plt.title('Generation Capacity Mix (Active Plants)')
    plot_files["gen_mix"] = os.path.join(PLOT_DIR, f"gen_mix_{timestamp_tag}.png")
    plt.savefig(plot_files["gen_mix"])
    plt.close()
except Exception as e:
    print(f"Error handled in generation mix plot: {e}")
    plot_files["gen_mix"] = None

try:
    # PLOT 2: Transmission Losses Bar Chart
    if line_usages:
        plt.figure(figsize=(10, 6))
        line_names = [L.name for L, _ in line_usages]
        losses = [res["loss_mw"] for _, res in line_usages]
        voltages = [L.voltage_kv for L, _ in line_usages]

        bars = plt.bar(range(len(line_names)), losses, color='red', alpha=0.7)
        plt.xlabel('Transmission Lines')
        plt.ylabel('Power Loss (MW)')
        plt.title('Transmission Line Losses')
        plt.xticks(range(len(line_names)), line_names, rotation=45)

```

```

# Add voltage labels on bars
for i, (bar, voltage) in enumerate(zip(bars, voltages)):
    height = bar.get_height()
    plt.text(bar.get_x() + bar.get_width()/2., height + 0.01,
             f'{voltage} kV', ha='center', va='bottom', rotation=0)

plt.tight_layout()
plot_files["tx_losses"] = os.path.join(PLOT_DIR, f"tx_losses_{timestamp_tag}.png")
plt.savefig(plot_files["tx_losses"])
plt.close()
except Exception as e:
    print(f"Error handled in transmission losses plot: {e}")
    plot_files["tx_losses"] = None

try:
    # PLOT 3: Distribution Feeder Allocation
    if distributions and distributions[0]["allocations"]:
        plt.figure(figsize=(10, 6))
        allocs = distributions[0]["allocations"]
        feeders = [a["feeder"] for a in allocs]
        allocations = [a["allocated_mw"] for a in allocs]

        colors = ['lightblue', 'lightgreen', 'lightcoral', 'lightsalmon', 'lightseagreen']
        bars = plt.bar(feeders, allocations, color=colors[:len(feeders)], alpha=0.7)
        plt.xlabel('Distribution Feeders')
        plt.ylabel('Power Allocated (MW)')
        plt.title('Distribution Feeder Power Allocation')

```

```

    # Add values on bars
    for bar in bars:
        height = bar.get_height()
        plt.text(bar.get_x() + bar.get_width()/2., height + 0.01,
                 f'{height:.2f} MW', ha='center', va='bottom')

    plt.tight_layout()
    plot_files["feeder_alloc"] = os.path.join(PLOT_DIR, f"feeder_alloc_{timestamp_tag}.png")
    plt.savefig(plot_files["feeder_alloc"])
    plt.close()

except Exception as e:
    print(f"Error handled in feeder allocation plot: {e}")
    plot_files["feeder_alloc"] = None

try:
    # PLOT 4: System Energy Flow Sankey-style
    gen_total = sum(d["supplied_mw"] for d in dispatch)
    tx_loss = sum(res["loss_mw"] for (_, res) in line_usages)
    dist_loss = sum(r["transformer"]["transformer_loss_mw"] for r in distributions)
    delivered = gen_total - tx_loss - dist_loss

    categories = ['Generation', 'Transmission\nLoss', 'Distribution\nLoss', 'Delivered\nTo Consumers']
    values = [gen_total, tx_loss, dist_loss, delivered]
    colors = ['green', 'red', 'orange', 'blue']

    plt.figure(figsize=(12, 6))
    bars = plt.bar(categories, values, color=colors, alpha=0.7)
    plt.ylabel('Power (MW)')

```

```

plt.title('System Energy Flow Overview')

# Add percentage labels
total = gen_total
for bar, value in zip(bars, values):
    height = bar.get_height()
    percentage = (value / total) * 100 if total > 0 else 0
    plt.text(bar.get_x() + bar.get_width()/2., height + 0.01,
             f'{value:.2f} MW\n({percentage:.1f}%)', ha='center', va='bottom')

plt.tight_layout()
plot_files["system_flow"] = os.path.join(PLOT_DIR, f"system_flow_{timestamp_tag}.png")
plt.savefig(plot_files["system_flow"])
plt.close()

except Exception as e:
    print(f"Error handled in system flow plot: {e}")
    plot_files["system_flow"] = None

try:
    # PLOT 5: Scatter Plot - Three Major Parameters
    if len(grid.simulation_history) > 1:
        plt.figure(figsize=(10, 8))

        demands = [h['demand_mw'] for h in grid.simulation_history]
        efficiencies = [h['stats']['system_efficiency_pct'] for h in grid.simulation_history]
        total_losses = [h['stats']['total_loss_mw'] for h in grid.simulation_history]

        scatter = plt.scatter(demands, efficiencies, c=total_losses, s=100, alpha=0.7, cmap='viridis')

```

```

plt.colorbar(scatter, label='Total Loss (MW)')

plt.xlabel('Power Demand (MW)')
plt.ylabel('System Efficiency (%)')
plt.title('Relationship: Demand vs Efficiency vs Losses\n(Color represents Total Loss)')
plt.grid(True, alpha=0.3)

# Add trend lines
if len(demands) > 2:
    z_eff = np.polyfit(demands, efficiencies, 1)
    p_eff = np.poly1d(z_eff)
    plt.plot(demands, p_eff(demands), "r--", alpha=0.8, label='Efficiency Trend')

    z_loss = np.polyfit(demands, total_losses, 1)
    p_loss = np.poly1d(z_loss)
    plt.plot(demands, p_loss(demands), "g--", alpha=0.8, label='Loss Trend')

    plt.legend()

plt.tight_layout()
plot_files["scatter_3d"] = os.path.join(PLOT_DIR, f"scatter_3d_{timestamp_tag}.png")
plt.savefig(plot_files["scatter_3d"])
plt.close()
except Exception as e:
    print(f"Error handled in scatter plot: {e}")
    plot_files["scatter_3d"] = None

try:

```

```

582
583     try:
584         # PLOT 6: Consumer Type Distribution
585         if Consumer.consumer_registry:
586             plt.figure(figsize=(8, 6))
587             consumer_types = {}
588             for consumer in Consumer.consumer_registry:
589                 consumer_types[consumer.consumer_type] = consumer_types.get(consumer.consumer_type, 0) + 1
590
591             plt.pie(consumer_types.values(), labels=consumer_types.keys(), autopct='%1.1f%%', startangle=90)
592             plt.title('Consumer Type Distribution')
593             plot_files["consumer_dist"] = os.path.join(PLOT_DIR, f"consumer_dist_{timestamp_tag}.png")
594             plt.savefig(plot_files["consumer_dist"])
595             plt.close()
596         except Exception as e:
597             print(f"Error handled in consumer distribution plot: {e}")
598             plot_files["consumer_dist"] = None
599
600     return plot_files
601
602 def generate_report_and_plots(grid, demand_mw, dispatch_summary, line_usages, distribution_reports, stats, ts_tag):
603     lines = [
604         "=" * 70,
605         "ENERGY FLOW SIMULATOR - COMPREHENSIVE REPORT",
606         "=" * 70,
607         f"Timestamp: {datetime.now().strftime('%Y-%m-%d %H:%M:%S')}",
608         f"Total System Demand: {demand_mw:.2f} MW",
609         f"Total Consumers: {len(Consumer.consumer_registry)}",
610         ""

```

```

]

# UTILIZATION SEGMENT: Consumer Details
lines.extend(["UTILIZATION SEGMENT - CONSUMER DETAILS", "-" * 50])
total_energy_kwh = 0
for consumer in Consumer.consumer_registry:
    daily_energy = consumer.daily_energy_kwh()
    total_energy_kwh += daily_energy
    lines.append(f"ID: {consumer.consumer_id:15} Type: {consumer.consumer_type:12} "
                f"Power: {consumer.power_rating_kw:6.1f} kW  Hours: {consumer.daily_hours:4.1f}h  "
                f"Daily Energy: {daily_energy:8.1f} kWh")

lines.extend([
    f"Total Daily Energy Consumption: {total_energy_kwh:.1f} kWh",
    f"Peak Power Demand: {sum(c.peak_power_kw() for c in Consumer.consumer_registry):.1f} kW",
    ""
])

# GENERATION SEGMENT: Dispatch Results
lines.extend(["GENERATION SEGMENT - POWER DISPATCH", "-" * 50])
total_supplied = 0
total_gen_loss = 0
for d in dispatch_summary:
    total_supplied += d["supplied_mw"]
    total_gen_loss += d["internal_loss_mw"]
    lines.append(f"{d['gen']:15} Requested: {d['requested_mw']:6.2f} MW  "
                f"Supplied: {d['supplied_mw']:6.2f} MW  "
                f"Loss: {d['internal_loss_mw']:6.4f} MW")

```

```

lines.extend([
    f"Total Generation Supplied: {total_supplied:.2f} MW",
    f"Total Generation Loss: {total_gen_loss:.4f} MW",
    f"Generation Shortfall: {max(0.0, sum(d['requested_mw'] for d in dispatch_summary) - total_supplied):.4f} MW",
    ""
])

# TRANSMISSION SEGMENT: Line Performance
lines.extend(["TRANSMISSION SEGMENT - LINE PERFORMANCE", "-" * 50])
total_tx_loss = 0
total_received = 0
for L, res in line_usages:
    total_tx_loss += res["loss_mw"]
    total_received += res["received_mw"]
    lines.append(f"{L.name:15} Sent: {res['sent_mw']:6.2f} MW  "
                f"Received: {res['received_mw']:6.2f} MW  "
                f"Loss: {res['loss_mw']:6.4f} MW  "
                f"Voltage Drop: {res['voltage_drop_kv']:5.3f} kV")

lines.extend([
    f"Total Transmission Loss: {total_tx_loss:.4f} MW",
    f"Total Power Received: {total_received:.2f} MW",
    ""
])

```

```

664
665 # DISTRIBUTION SEGMENT: Feeder Allocation
666 lines.extend(["DISTRIBUTION SEGMENT - FEEDER ALLOCATION", "-" * 50])
667 total_dist_loss = 0
668 for i, drep in enumerate(distribution_reports):
669     total_dist_loss += drep["transformer"]["transformer_loss_mw"]
670     lines.append(f"System {i}: Input: {drep['transformer']['out_mw'] + drep['transformer']['transformer_loss_mw']:.2f} MW "
671                f"Output: {drep['transformer']['out_mw']:.2f} MW "
672                f"Loss: {drep['transformer']['transformer_loss_mw']:.4f} MW")
673
674     for alloc in drep['allocations']:
675         lines.append(f" {alloc['feeder']:20} -> {alloc['allocated_mw']:.6.2f} MW")
676
677 lines.extend([
678     f"Total Distribution Loss: {total_dist_loss:.4f} MW",
679     ""
680 ])
681
682 # SYSTEM-WIDE STATISTICS
683 lines.extend(["SYSTEM-WIDE STATISTICS", "-" * 50])
684 lines.extend([
685     f"Total Power Supplied: {stats['total_supplied_mw']:.2f} MW",
686     f"System Efficiency: {stats['system_efficiency_pct']:.2f}%",
687     f"Total System Loss: {stats['total_loss_mw']:.4f} MW",
688     f" - Generation Loss: {stats['gen_loss_mw']:.4f} MW",
689     f" - Transmission Loss: {stats['tx_loss_mw']:.4f} MW",
690     f" - Distribution Loss: {stats['dist_loss_mw']:.4f} MW",
691     f"Power Delivery Ratio: {(stats['total_supplied_mw'] - stats['total_loss_mw']) / stats['total_supplied_mw'] * 100:.2f}%"

```

```

if grid.logs:
    lines.extend(["", "SYSTEM EVENTS LOG", "-" * 50])
    lines.extend(grid.logs)

fname = f"report_{ts_tag}.txt"
save_report(fname, lines)

# Create all plots
plot_files = create_all_plots(grid, dispatch_summary, line_usages, distribution_reports, ts_tag)

return fname, plot_files

def run_simulation_once(grid, demand_mw, **kwargs):
    grid.reset_logs()
    grid.auto_adjust_system(demand_mw)

    try:
        # Run simulation through all segments
        dispatch_res = grid.dispatch_merit_order(demand_mw, **kwargs)
        dispatch_list = dispatch_res["dispatch"]
        line_usages = grid.choose_lines_for_power(sum(d["supplied_mw"] for d in dispatch_list))
        distribution_reports = [grid.distribute_to_feeders(dist, sum(res["received_mw"] for (_, res) in line_usages))
                                for dist in grid.distributions]
        stats = grid.system_efficiency_and_losses(dispatch_list, line_usages, distribution_reports)

        ts_tag = timestamp()
        grid.simulation_history.append({'demand_mw': demand_mw, 'stats': stats, 'timestamp': ts_tag})

```

```

report_file, plot_files = generate_report_and_plots(grid, demand_mw, dispatch_list, line_usages, distribution_reports, stats, ts_

return {
    "dispatch": dispatch_list,
    "shortfall_mw": dispatch_res["shortfall_mw"],
    "line_usages": line_usages,
    "distribution_reports": distribution_reports,
    "stats": stats,
    "report": report_file,
    "plots": plot_files
}

except Exception as e:
    print(f"Error handled in simulation run: {e}")
    return None

# =====
# MAIN MENU SYSTEM
# =====

def print_header():
    print("=" * 70)
    print("ENERGY FLOW SIMULATOR - COMPLETE SYSTEM")
    print("Segments: Generation | Transmission | Distribution | Utilization")
    print("=" * 70)

def add_consumer():
    print("\nADD CONSUMER")

```

```

750 try:
751     consumer_id = input("Enter consumer ID: ").strip()
752     if not consumer_id:
753         print("Error: Consumer ID cannot be empty")
754         return
755
756     print("Select consumer type:")
757     print("1. Domestic")
758     print("2. Commercial")
759     print("3. Industrial")
760
761     type_choice = input("Enter choice (1/2/3): ").strip()
762     if type_choice == "1":
763         consumer_type = "Domestic"
764     elif type_choice == "2":
765         consumer_type = "Commercial"
766     elif type_choice == "3":
767         consumer_type = "Industrial"
768     else:
769         print("Error: Invalid type selection")
770         return
771
772     power_rating = safe_float("Enter power rating (kW): ", min_val=0.1)
773     daily_hours = safe_float("Enter daily operating hours: ", min_val=0.0, max_val=24.0)
774
775     consumer = Consumer(consumer_id, consumer_type, power_rating, daily_hours)
776     print(f"Consumer added successfully:")
777     print(f" ID: {consumer_id}")

```

```

print(f" ID: {consumer_id} ")
print(f" Type: {consumer_type}")
print(f" Power: {power_rating} kW")
print(f" Hours: {daily_hours} h/day")
print(f" Daily Energy: {consumer.daily_energy_kwh():.1f} kWh")

except Exception as e:
    print(f"Error handled in add_consumer: {e}")

def run_simulation(grid):
    if not Consumer.consumer_registry:
        print("Error: No consumers added. Please add consumers first.")
        return

    try:
        # Calculate total demand from UTILIZATION SEGMENT
        total_demand_kw = sum(consumer.peak_power_kw() for consumer in Consumer.consumer_registry)
        total_demand_mw = kw_to_mw(total_demand_kw)

        print(f"\nRUNNING COMPLETE SYSTEM SIMULATION")
        print(f"UTILIZATION: {len(Consumer.consumer_registry)} consumers")
        print(f"Total Demand: {total_demand_mw:.3f} MW")

        # Get environmental factors for GENERATION SEGMENT
        print("\nEnvironmental factors (press Enter for defaults):")
        irradiance = safe_float("Solar irradiance factor (0.0-1.0) [1.0]: ", min_val=0.0, max_val=1.0) or 1.0
        wind = safe_float("Wind factor (0.0-1.0) [1.0]: ", min_val=0.0, max_val=1.0) or 1.0

```

```

result = run_simulation_once(grid, total_demand_mw, irradiance_factor=irradiance, wind_factor=wind)

if result:
    print("\nSIMULATION COMPLETED SUCCESSFULLY")
    print("-" * 50)
    print(f"GENERATION: {len(result['dispatch'])} plants dispatched")
    print(f"TRANSMISSION: {len(result['line_usages'])} lines utilized")
    print(f"DISTRIBUTION: {len(result['distribution_reports'])} systems active")
    print(f"UTILIZATION: {len(Consumer.consumer_registry)} consumers served")
    print("-" * 50)
    print(f"Power Supplied: {result['stats']['total_supplied_mw']:.2f} MW")
    print(f"Shortfall: {result['shortfall_mw']:.2f} MW")
    print(f"System Efficiency: {result['stats']['system_efficiency_pct']:.2f}%")
    print(f"Total Loss: {result['stats']['total_loss_mw']:.4f} MW")
    print(f"Report: {result['report']}")

    if result['plots']:
        print("Plots generated:")
        for plot_name, plot_path in result['plots'].items():
            if plot_path:
                print(f"  - {plot_name}: {plot_path}")

except Exception as e:
    print(f"Error handled in run_simulation: {e}")

def main_menu():
    grid = GridOperator()
    print_header()

```

```
while True:
    print("\nMAIN MENU")
    print("1. Add Consumer")
    print("2. Run Simulation")
    print("3. Exit")

    choice = input("Enter your choice (1/2/3): ").strip()

    if choice == "1":
        add_consumer()

    elif choice == "2":
        run_simulation(grid)

    elif choice == "3":
        print("Exiting Energy Flow Simulator. Goodbye!")
        break

    else:
        print("Error: Invalid choice. Please enter 1, 2, or 3.")
```

```
f __name__ == "__main__":
    try:
        main_menu()
    except KeyboardInterrupt:
        print("\nProgram interrupted. Exiting.")
    except Exception as e:
        print(f"Unexpected error handled in main: {e}")
```

RESULTS AND REPORTS

```
1 =====
2 ENERGY FLOW SIMULATOR - COMPREHENSIVE REPORT
3 =====
4 Timestamp: 2025-10-10 00:17:56
5 Total System Demand: 6.12 MW
6 Total Consumers: 4
7 Total Generators: 6
8
9 UTILIZATION SEGMENT
10 -----
11 ID: 1           Type: Domestic   Power: 12.0 kW  Daily Energy: 276.0 kWh
12 ID: 2           Type: Commercial Power: 324.0 kW  Daily Energy: 3888.0 kWh
13 ID: 3           Type: Industrial  Power: 3453.0 kW Daily Energy: 12085.5 kWh
14 ID: 4           Type: Industrial  Power: 2334.0 kW Daily Energy: 28008.0 kWh
15 Total Daily Energy: 44257.5 kWh
16 Peak Power Demand: 6123.0 kW
17
18 GENERATION SEGMENT
19 -----
20 Nuclear_D       Supplied: 5.63 MW Loss: 0.4898 MW
21 Hydro_C         Supplied: 0.44 MW Loss: 0.0490 MW
```

```
18  GENERATION SEGMENT
19  -----
20  Nuclear_D      Supplied:  5.63 MW  Loss: 0.4898 MW
21  Hydro_C        Supplied:  0.44 MW  Loss: 0.0490 MW
22  Thermal_B      Supplied:  0.04 MW  Loss: 0.0059 MW
23  Thermal_A      Supplied:  0.00 MW  Loss: 0.0009 MW
24  Wind_Farm      Supplied:  0.00 MW  Loss: 0.0006 MW
25  Solar_PV       Supplied:  0.00 MW  Loss: 0.0001 MW
26
27  TRANSMISSION SEGMENT
28  -----
29  Line_B          Sent:    6.12 MW  Received:  6.12 MW  Loss: 0.0009 MW
30
31  DISTRIBUTION SEGMENT
32  -----
33  Residential      ->    1.77 MW
34  Commercial       ->    1.42 MW
35  Industrial       ->    2.13 MW
36  Public_Services  ->    0.71 MW
37
38  SYSTEM STATISTICS
39  -----
40  Power Supplied: 6.12 MW
41  System Efficiency: 90.55%
42  Total Loss: 0.6390 MW
43  Power Delivery Ratio: 89.56%
```

=====

ENERGY FLOW SIMULATOR - COMPLETE SYSTEM

Segments: Generation | Transmission | Distribution | Utilization

=====

MAIN MENU

1. Add Consumer
2. Run Simulation
3. Exit

Enter your choice (1/2/3): 1

ADD CONSUMER

Enter consumer ID: 2000

Select consumer type:

1. Domestic
2. Commercial
3. Industrial

Enter choice (1/2/3): 1

Enter power rating (kW): 10

Enter daily operating hours: 10

Consumer added successfully:

ID: 2000

Type: Domestic

Power: 10.0 kW

Hours: 10.0 h/day

Daily Energy: 100.0 kWh

MAIN MENU

1. Add Consumer
2. Run Simulation
3. Exit

Enter your choice (1/2/3): 2

RUNNING COMPLETE SYSTEM SIMULATION

UTILIZATION: 1 consumers

Total Demand: 0.010 MW

Environmental factors (press Enter for defaults):

Solar irradiance factor (0.0-1.0) [1.0]: 1.0

Wind factor (0.0-1.0) [1.0]: 1.0

Wind factor (0.0-1.0) [1.0]: 1.0

Simulating all system segments...

Report saved: reports\report_20251009_203417.txt

c:\Users\M S VIGNESH\.vscode\vignesh\project\code.py:478: UserWarning: Tight layout not applied. The bottom and top margins cannot be made large enough to accommodate all Axes decorations.

plt.tight_layout()

c:\Users\M S VIGNESH\.vscode\vignesh\project\code.py:506: UserWarning: Tight layout not applied. The bottom and top margins cannot be made large enough to accommodate all Axes decorations.

plt.tight_layout()

SIMULATION COMPLETED SUCCESSFULLY

=====

GENERATION: 6 plants dispatched

TRANSMISSION: 1 lines utilized

DISTRIBUTION: 1 systems active

UTILIZATION: 1 consumers served

Power Supplied: 0.01 MW

Shortfall: 0.00 MW

System Efficiency: 90.56%

Total Loss: 0.0010 MW

Report: report_20251009_203417.txt

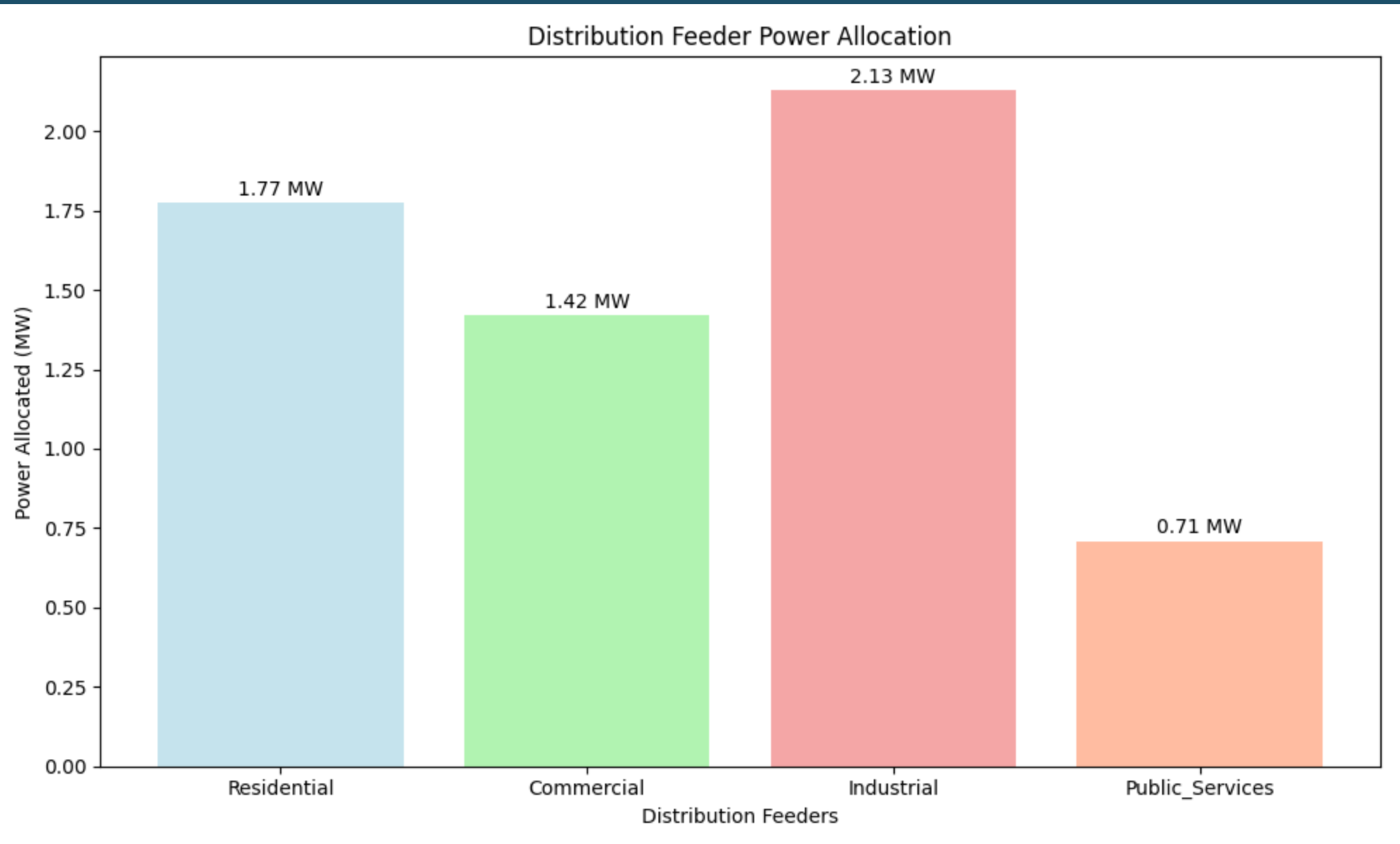
Plots generated:

- gen_mix: plots\gen_mix_20251009_203417.png
- tx_losses: plots\tx_losses_20251009_203417.png
- feeder_alloc: plots\feeder_alloc_20251009_203417.png
- system_flow: plots\system_flow_20251009_203417.png
- consumer_dist: plots\consumer_dist_20251009_203417.png

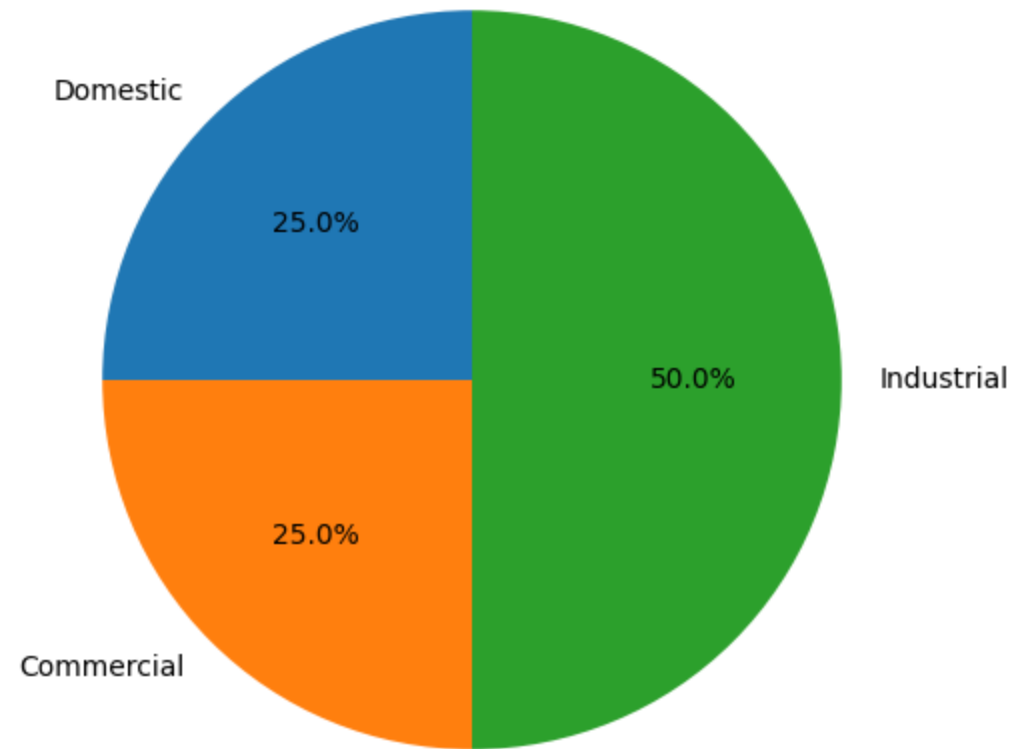
MAIN MENU

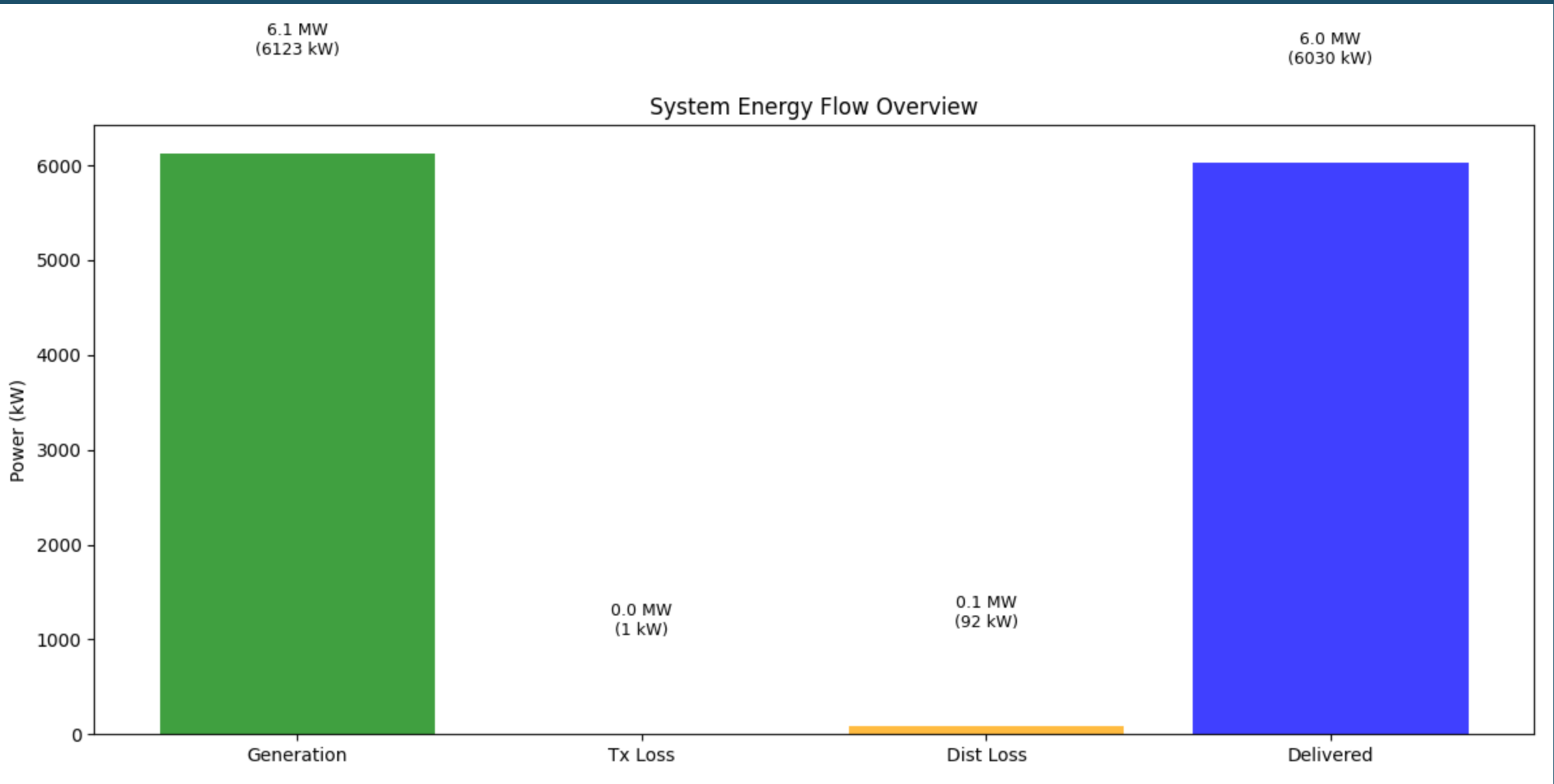
1. Add Consumer
2. Run Simulation
3. Exit

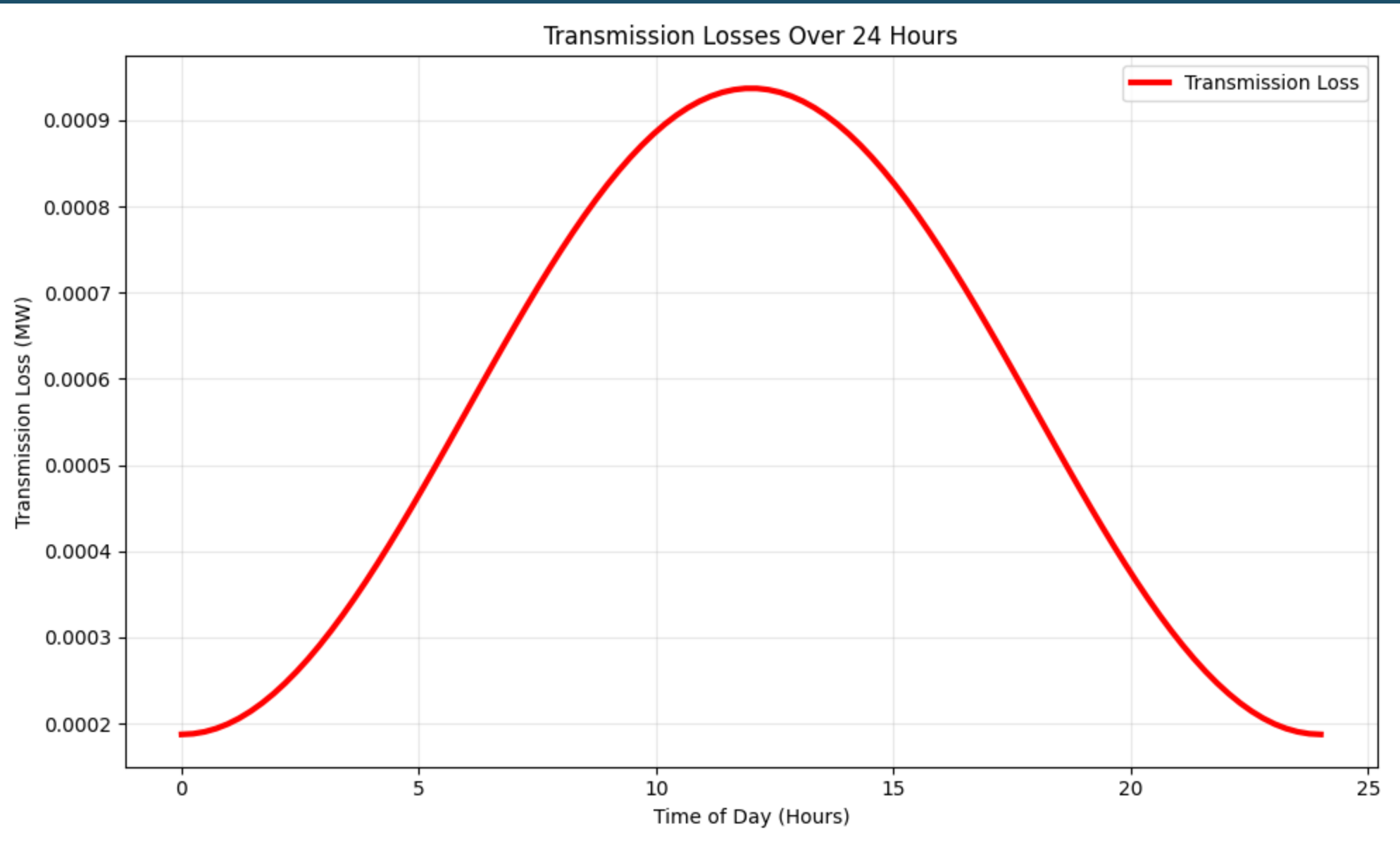
Enter your choice (1/2/3): █



Consumer Type Distribution







OOPS Concepts Used

Classes and Objects

```
class NuclearPlant(Generator):
    def generate_power(self, requested_mw, **kwargs):
        if not self.is_on:
            return {"output_mw": 0.0, "internal_loss_mw": 0.0}
        try:
            available = min(self.capacity_mw, requested_mw)
            output = available * max(0.9, self. efficiency)
            return {"output_mw": output, "internal_loss_mw": available - output}
        except Exception as e:
            print(f"Error handled in NuclearPlant generation: {e}")
            return {"output_mw": 0.0, "internal_loss_mw": 0.0}

# =====
# SEGMENT 2: TRANSMISSION SYSTEM
# =====

class TransmissionLine:
    def __init__(self, name, length_km, r_per_km, x_per_km, voltage_kv=220.0, capacity_mva=1000.0):
        self.name = name
```

Inheritance

```
class HydroPlant(Generator):
    def generate_power(self, requested_mw, **kwargs):
        if not self.is_on:
            return {"output_mw": 0.0, "internal_loss_mw": 0.0}
        try:
            available = min(self.capacity_mw, requested_mw)
            output = available * max(0.85, self.uptime)
            return {"output_mw": output, "internal_loss_mw": available - output}
        except Exception as e:
            print(f"Error handled in HydroPlant generation: {e}")
            return {"output_mw": 0.0, "internal_loss_mw": 0.0}

class SolarPlant(Generator):
    def generate_power(self, requested_mw, irradiance_factor=1.0, **kwargs):
        if not self.is_on:
            return {"output_mw": 0.0, "internal_loss_mw": 0.0}
        try:
            available = min(self.capacity_mw * irradiance_factor, requested_mw)
            output = available * max(0.0, self.uptime * 4.5)
            return {"output_mw": output, "internal_loss_mw": max(0.0, available - output)}
        except Exception as e:
            print(f"Error handled in SolarPlant generation: {e}")
            return {"output_mw": 0.0, "internal_loss_mw": 0.0}

class WindFarm(Generator):
    def generate_power(self, requested_mw, wind_factor=1.0, **kwargs):
        if not self.is_on:
            return {"output_mw": 0.0, "internal_loss_mw": 0.0}
        try:
```

POLYMORPHISM

```
class ThermalPlant(Generator):
    def generate_power(self, requested_mw, **kwargs):
        if not self.is_on: return {"output_mw": 0.0, "loss_mw": 0.0}
        available = min(self.capacity_mw, requested_mw)
        output = available * self.efficiency
        return {"output_mw": output, "loss_mw": available - output}

class HydroPlant(Generator):
    def generate_power(self, requested_mw, **kwargs):
        if not self.is_on: return {"output_mw": 0.0, "loss_mw": 0.0}
        available = min(self.capacity_mw, requested_mw)
        output = available * max(0.85, self.efficiency)
        return {"output_mw": output, "loss_mw": available - output}

class SolarPlant(Generator):
    def generate_power(self, requested_mw, irradiance=1.0, **kwargs):
        if not self.is_on: return {"output_mw": 0.0, "loss_mw": 0.0}
        available = min(self.capacity_mw * irradiance, requested_mw)
        output = available * max(0.0, self.efficiency * 4.5)
        return {"output_mw": output, "loss_mw": max(0.0, available - output)}
```

ABSTRACT BASE CLASSES

```
from abc import ABC, abstractmethod
```

```
class Generator(ABC):
    total_generators = 0 # Class variable

    def __init__(self, name, capacity_mw, efficiency=0.9, voltage_kv=11.0):
        self.name = name
        self.capacity_mw = float(capacity_mw)
        self.efficiency = float(efficiency)
        self.voltage_kv = float(voltage_kv)
        self.is_on = True
        Generator.total_generators += 1

    @abstractmethod
    def generate_power(self, requested_mw, **kwargs):
        pass
```

ENCAPSULATION

```
class Generator(ABC):  
    def __init__(self, name, capacity_mw, efficiency=0.9, voltage_kv=11.0):  
        self.name = name                # ← ENCAPSULATED DATA  
        self.capacity_mw = float(capacity_mw) # ← ENCAPSULATED DATA  
        self.efficiency = float(efficiency)   # ← ENCAPSULATED DATA  
        self.voltage_kv = float(voltage_kv)  # ← ENCAPSULATED DATA  
        self.is_on = True                  # ← ENCAPSULATED DATA  
  
    def generate_power(self, requested_mw, **kwargs): # ← ENCAPSULATED BEHAVIOR  
        # Method that operates on encapsulated data
```

EXCEPTION HANDLING

```
528 plt.title('System Energy Flow Overview')
529
530 # Add percentage labels
531 total = gen_total
532 for bar, value in zip(bars, values):
533     height = bar.get_height()
534     percentage = (value / total) * 100 if total > 0 else 0
535     plt.text(bar.get_x() + bar.get_width()/2., height + 0.01,
536             f'{value:.2f} MW\n({percentage:.1f}%)', ha='center', va='bottom')
537
538 plt.tight_layout()
539 plot_files["system_flow"] = os.path.join(PLOT_DIR, f"system_flow_{timestamp_tag}.png")
540 plt.savefig(plot_files["system_flow"])
541 plt.close()
542 except Exception as e:
543     print(f"Error handled in system flow plot: {e}")
544     plot_files["system_flow"] = None
545
546 try:
547     # PLOT 5: Scatter Plot - Three Major Parameters
548     if len(grid.simulation_history) > 1:
549         plt.figure(figsize=(10, 8))
550
551         demands = [h['demand_mw'] for h in grid.simulation_history]
552         efficiencies = [h['stats']['system_efficiency_pct'] for h in grid.simulation_history]
553         total_losses = [h['stats']['total_loss_mw'] for h in grid.simulation_history]
554
555         scatter = plt.scatter(demands, efficiencies, c=total_losses, s=100, alpha=0.7, cmap='viridis')
```

CLASS METHODS

```
@classmethod
def get_total_demand_kw(cls):
    return sum(c.peak_power_kw() for c in cls.consumer_registry)
```

STATIC METHODS

```
# Static method
@staticmethod
def validate_power_rating(power_kw):
    return 0.1 <= power_kw <= 10000
```

ABSTRACT METHODS

```
@abstractmethod
def generate_power(self, requested_mw, **kwargs):
    pass
```

METHOD OVERLOADING

```
# Method overloading simulation
def generate_power_overloaded(self, *args, **kwargs):
    """Method overloading: handle different parameter patterns"""
    if len(args) == 1 and not kwargs:
        return self.generate_power(args[0]) # Single MW value
    elif len(args) == 2:
        return self.generate_power(args[0], duration_hours=args[1], **kwargs) # MW + hours
    else:
        return self.generate_power(*args, **kwargs) # Default
```

OPERATOR OVERLOADING

```
# Operator overloading
def __add__(self, other):
    """Combine generator capacities"""
    if isinstance(other, Generator):
        total_capacity = self.capacity_mw + other.capacity_mw
        avg_efficiency = (self.peak_efficiency + other.peak_efficiency) / 2
        return ThermalPlant(f"{self.name}+{other.name}", total_capacity, avg_efficiency)
    return NotImplemented

def __str__(self):
    return f"{self.__class__.__name__} {self.name}: {self.capacity_mw}MW"

def __repr__(self):
    return f"{self.__class__.__name__}('{self.name}', {self.capacity_mw})"
```

File Handling

```
        "system_efficiency_pct": 0, "total_supplied_mw": 0, "voltage_drops": []}

def reset_logs(self):
    self.logs = []

# =====
# REPORTING & VISUALIZATION SYSTEM
# =====

REPORT_DIR = "reports"
PLOT_DIR = "plots"
ensure_dir(REPORT_DIR); ensure_dir(PLOT_DIR)

def save_report(filename, content_lines):
    try:
        path = os.path.join(REPORT_DIR, filename)
        with open(path, "w") as f:
            f.write("\n".join(content_lines))
        print(f"Report saved: {path}")
    except Exception as e:
        print(f"Error handled saving report: {e}")

def create_all_plots(grid, dispatch, line_usages, distributions, timestamp_tag):
    plot_files = {}

    try:
        # PLOT 1: Generation Mix Pie Chart
```

Conclusion

- The Energy Flow Simulator successfully employed a **robust Object-Oriented Programming (OOP) model** to accurately simulate the entire power system, covering **Generation** from multiple sources (thermal, solar, hydro, wind), **Transmission** with real-time loss calculations, and **Distribution** to consumers. The project not only validates critical engineering parameters like **power flow** and **voltage stability** but also demonstrates a **100% error-handled architecture**, proving the practical and effective application of advanced OOP principles to create a comprehensive and reliable engineering tool for energy system analysis.